

SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 2 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO2 – Precision (BL3)	Assessment:	Debugging and Optimisation task
Weightage:	45%	Total Marks:	100
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/nonrestoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems.		
Student Name:	Deepan Kumar P	Reg. No.:	192312498
Section / Batch:	Final year ECE	Date:	03/08/2026

Instructions to Students

- Identify arithmetic errors and performance bottlenecks in the given problem.
- Analyse the execution steps to locate the source of errors.
- Debug the algorithm by correcting logical and computational faults.
- Optimize the solution using appropriate arithmetic techniques or algorithms.
- Evaluate the optimized solution for correctness, accuracy, and efficiency.

QUESTIONS

1. A computer system performing signed binary subtraction using 2's complement produces incorrect results for certain operand combinations. Debug the subtraction process by examining binary conversion, complement generation, and overflow detection. Suggest appropriate corrections to ensure accurate signed arithmetic.
2. A digital processor implementing a carry look-ahead adder fails to produce the correct sum for some input combinations. Debug the generate and propagate logic to identify the source of the error. Recommend modifications that restore correct operation while maintaining high-speed performance.
3. A hardware multiplier using Booth's algorithm consumes more clock cycles than expected during signed multiplication. Debug the execution sequence by analyzing bit-pair decisions, arithmetic shifts, and register updates. Propose optimization techniques to improve execution efficiency without affecting accuracy.
4. A processor implementing the non-restoring division algorithm produces incorrect quotient values for specific dividend and divisor combinations. Debug the step-by-step division process by examining partial remainders, shifting operations, and quotient-bit generation. Suggest corrections to obtain accurate division results.
5. A graphics processing application experiences performance degradation due to frequent floatingpoint computations using IEEE 754 double-precision representation. Debug the arithmetic operations to identify unnecessary precision or computational overhead. Recommend suitable optimization techniques to improve execution speed while maintaining the required numerical accuracy.

Code of Conduct

I Deepan Kumar P (192312498)certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Deepan Kumar P

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Identification of Errors / Bugs	20	Accurately identifies all errors and issues with complete understanding of the problem	Identifies most errors with minor omissions	Identifies limited errors; misses key issues	Unable to identify errors or misunderstands the problem
Debugging Approach & Analysis	20	Uses effective debugging techniques with clear analysis and root cause identification	Applies suitable debugging methods with minor gaps	Uses basic debugging methods with limited analysis	No systematic debugging approach followed
Correctness of Debugged Solution	25	Provides completely corrected solution with accurate functionality and expected output	Provides working solution with minor errors	Partially correct solution with limited functionality	Incorrect solution or fails to resolve errors
Optimization Techniques Applied	20	Applies efficient optimization methods to improve performance, memory usage and quality	Performs optimization with noticeable improvements	Limited optimization with minimal improvements	No optimization applied or inefficient implementation
Testing and Documentation	15	Performs complete testing with proper validation and clear documentation	Provides adequate testing and documentation with minor issues	Limited testing and incomplete documentation	No proper testing or documentation provided

Marks Evaluation:

Q. No	Identification of Errors / Bugs (20)	Debugging Approach & Analysis (20)	Correctness of Debugged Solution (25)	Optimization Techniques Applied (20)	Testing and Documentation (15)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	/ 20	/ 25	/ 20	/ 15	/ 100

Course Coordinator

Faculty Incharge

1 . Debugging 2's Complement Subtraction

Given Problem:

The processor produces incorrect results during signed subtraction using 2's complement.

What the Question Means:

You must identify where the subtraction process is failing by checking binary conversion, 2's complement generation and overflow detection.

Possible Errors:

- Incorrect decimal-to-binary conversion.
- Wrong 2's complement generation (forgot +1).
- Direct subtraction instead of $A + 2's\ complement(B)$.
- Overflow not detected.

Debugging:

1. Verify binary conversion.
2. Verify 2's complement.
3. Perform subtraction using binary addition.
4. Ignore final carry.
5. Check overflow.

Corrections:

- Generate correct 2's complement.
- Use binary adder for subtraction.
- Add overflow detection logic.

Optimisation:

- Carry Look-Ahead Adder.
- Hardware complement generator.
- Automatic overflow detection.

Testing:

Test with positive, negative and overflow cases to validate correctness.

Conclusion:

Correct conversion, complement generation and overflow detection restore accurate signed subtraction.

2 . Debugging Carry Look-Ahead Adder

Given Problem:

The Carry Look-Ahead Adder (CLA) produces incorrect sums for some inputs.

What the Question Means:

Find errors in Generate (G), Propagate (P) and carry logic while preserving high speed.

Possible Errors:

- Wrong Generate signal.
- Wrong Propagate signal.
- Incorrect carry equations.
- Faulty gate connections.

Debugging:

1. Verify $G=A \cdot B$.
2. Verify $P=A \oplus B$.
3. Check carry equations.
4. Validate final sum bits.

Corrections:

Repair carry logic and gate connections.

Optimisation:

- Hierarchical CLA.
- Reduced fan-out.
- Optimised gate delay.

Testing:

Verify with truth tables and multiple binary additions.

Conclusion:

Correct G and P logic restores fast and accurate addition.

Question 3 – Debugging Booth's Algorithm

Given Problem:

Booth multiplier requires more clock cycles than expected.

What the Question Means:

Find why multiplication is slow by checking bit-pair decisions, arithmetic shifts and register updates.

Possible Errors:

- Wrong Booth decision.
- Incorrect arithmetic right shift.
- Wrong register update.
- Extra unnecessary cycles.

Debugging:

1. Check Q0 and Q-1.
2. Verify add/subtract operation.
3. Verify arithmetic right shift.
4. Update registers correctly.

Corrections:

Apply Booth rules correctly each iteration.

Optimisation:

- Radix-4 Booth encoding.
- Skip unnecessary additions.
- Faster register control.

Testing:

Test positive, negative and mixed-sign multiplication.

Conclusion:

Correct Booth execution reduces clock cycles while maintaining accuracy.

4 .Debugging Non-Restoring Division

Given Problem:

The processor produces incorrect quotient values using non-restoring division.

What the Question Means:

Identify errors in partial remainder calculation, shifts and quotient generation.

Possible Errors:

- Incorrect partial remainder.
- Wrong shift operation.
- Wrong quotient bit.
- Incorrect final remainder correction.

Debugging:

1. Verify subtraction/addition.
2. Verify left shift.
3. Check quotient bits.
4. Correct final remainder if required.

Corrections:

Follow the non-restoring algorithm correctly.

Optimisation:

- Dedicated shift hardware.
- Pipeline divider.
- Reduce unnecessary operations.

Testing:

Check quotient and remainder using different dividend/divisor values.

Conclusion:

Correct remainder handling produces accurate quotient and faster division.

5. Debugging IEEE 754 Floating Point

Given Problem:

Double-precision floating-point operations reduce graphics application performance.

What the Question Means:

Determine whether unnecessary precision and floating-point overhead are slowing execution.

Possible Errors:

- Unnecessary double precision.
- Incorrect exponent alignment.
- Incorrect normalisation.
- Incorrect rounding.

Debugging:

1. Verify sign, exponent and mantissa.
2. Align exponents.
3. Perform arithmetic.
4. Normalise and round.

Corrections:

Use correct IEEE 754 arithmetic and only the precision actually required.

Optimisation:

- Use single precision where sufficient.
- SIMD/vector instructions.
- Fused multiply-add.
- Reduce unnecessary conversions.

Testing:

Compare outputs with IEEE 754 reference values and verify acceptable numerical accuracy.

Conclusion:

Reducing unnecessary precision improves execution speed while maintaining required accuracy.
